

Name of the Experiment

LAB EXPERIMENT WEEK - 1 Installation of Java and Eclipse & Simple, if-else and loops

Date :

Ex. No. :

Aim: Use Eclipse or Net bean platform and acquaint with the various menus. Create a test project, add a test class, and run it. See how you can use auto suggestions, auto fill. Try code formatter and code refactoring like renaming variables, methods, and classes. Try debug step by step with a small program of about 10 to 15 lines which contains at least one if else condition and a for loop.

Solution:

- **Step 1** - Install JDK in the computer.
- **Step 2** - Set the path in the Environment Variables from Advanced Setting of computer
- **Step 3** - Download Eclipse from Eclipse website
- **Step 4** - Install the Eclipse (follow the screen to install eclipse)



- Select the suitable version based on your OS, then download get starts.



- Double click on the Eclipse Application.

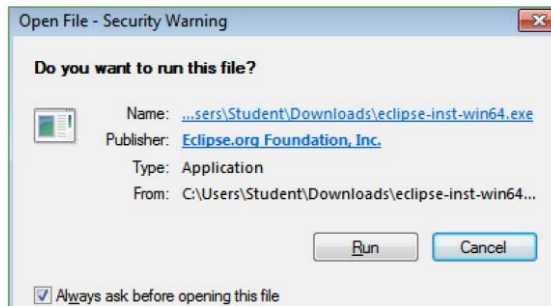
DQ_List	3/27/2019 9:27 AM	Microsoft Office E...	10 KB
eclipse-inst-win64	12/17/2019 2:56 PM	Application	53,427 KB
Exercise1	Date created: 12/17/2019 2:39 PM	JAVA File	3 KB
img028	Size: 52.1 MB	JPEG image	21 KB
Installer_microsoft_office_2007314_02421...	5/28/2019 11:37 AM	Application	2,102 KB

Name of the
Experiment

Date :

Ex. No. :

- Click on Run in the Security Warning box.



- Then, the installation process begins.



- Click on Eclipse IDE for Java Developers.



- Click on Install button.



Name of the
Experiment

Date :

Ex. No. :

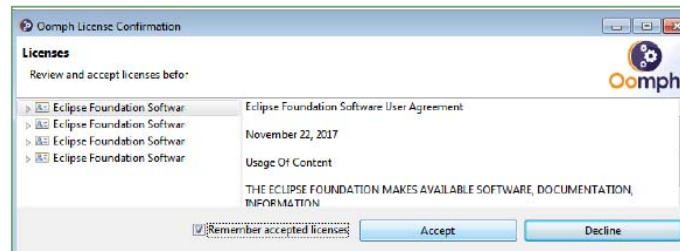
- Click on Accept Now.



- Then the Eclipse installation begins.



- Click on Accept



- Click on Select All and Accept Selected.

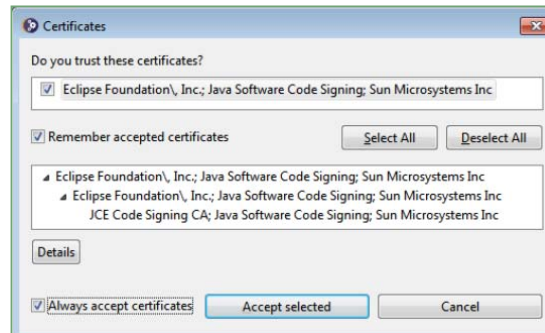


Name of the
Experiment

Date :

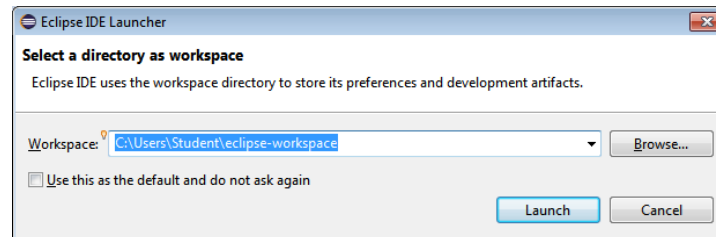
Ex. No. :

- After completing, click on Launch to start the Eclipse IDE.



CREATING PROJECT AND CLASSES IN ECLIPSE IDE

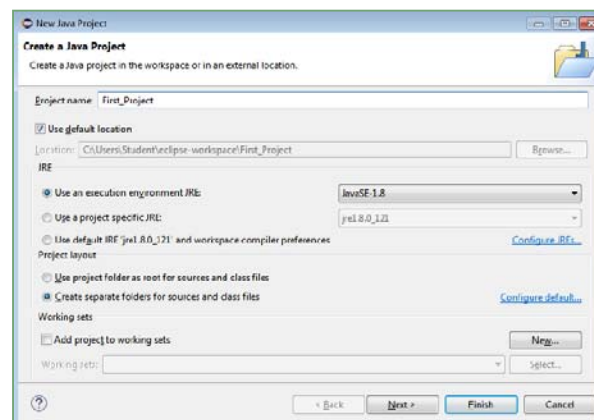
- Browse the Workspace for storing the java project and click on Launch.



- Select "Create a new Java project".



- Type the project name and click on Finish.

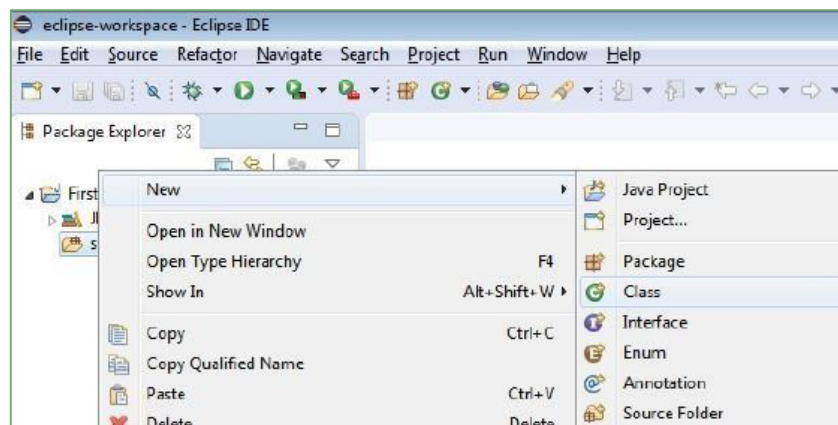
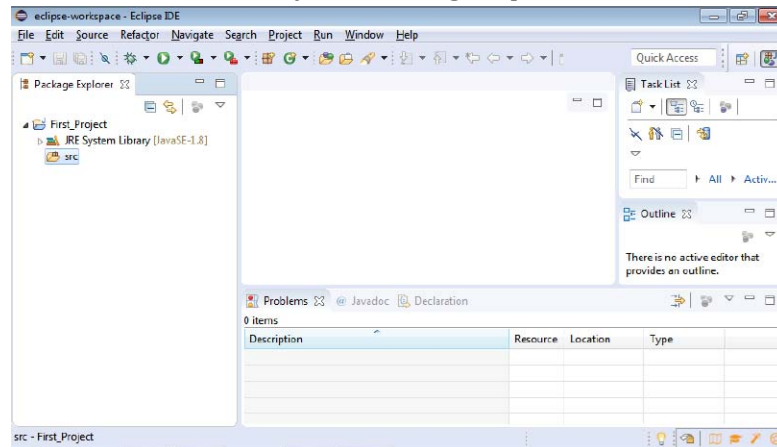


Name of the
Experiment

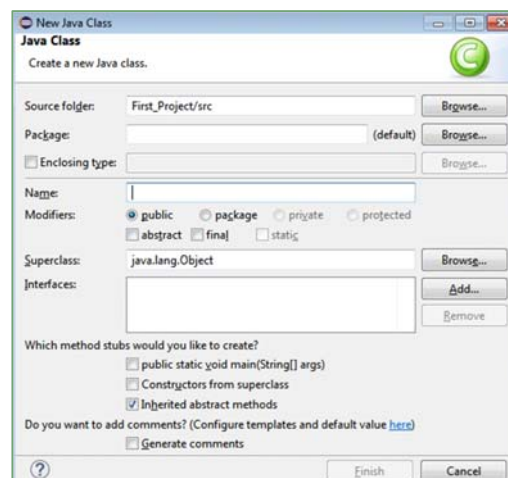
Date :

Ex. No. :

- Now, create the class in src directory from Package Explorer window.



- Type the class name and click on Finish.

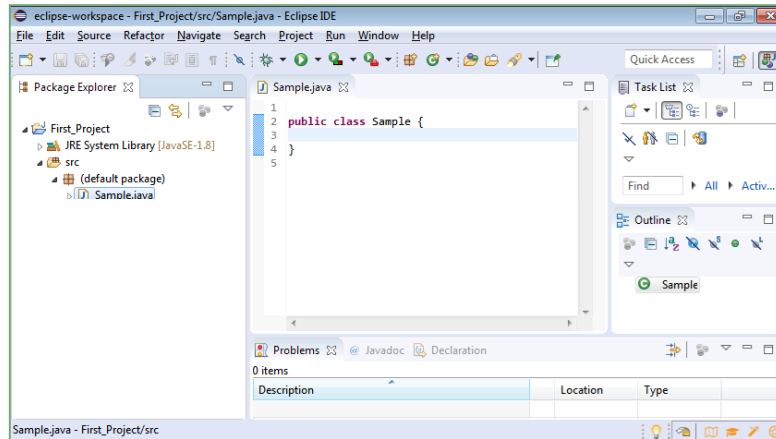


Name of the
Experiment

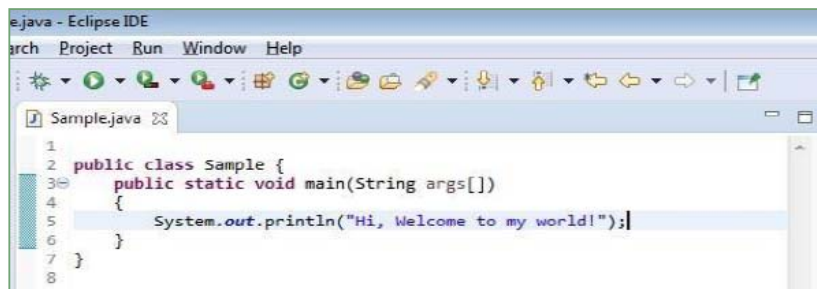
Date :

Ex. No. :

- Type the java code.



- Click on Play button to run or execute the java code.



Program Using If and For Loop

```
public class Prog1{
    public static void main(String[] args){
        System.out.println("\n Prog. is showing even no");
        for(int i=2;i<=20;i++){
            if(i%2==0) {
                System.out.print("\n "+i);
            }
        }
    }
}
```

Commands

Compile

D:>javac Prog1.java

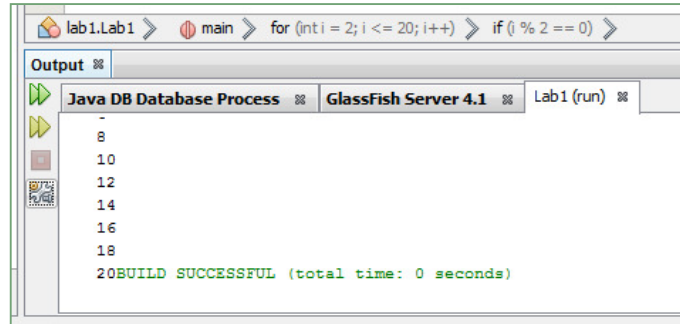
Run

D:>java Prog1

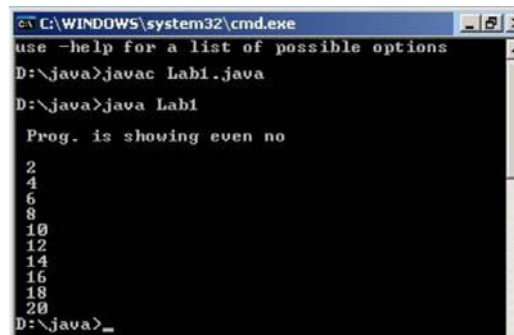
Name of the
Experiment

Date :

Ex. No. :

Output**In Netbeans IDE**

```
lab1.Lab1 > main > for (inti = 2; i <= 20; i++) > if (i % 2 == 0) >  
Output  
Java DB Database Process GlassFish Server 4.1 Lab1 (run)  
2  
4  
6  
8  
10  
12  
14  
16  
18  
20  
BUILD SUCCESSFUL (total time: 0 seconds)
```

In Command Prompt

```
C:\WINDOWS\system32\cmd.exe  
use -help for a list of possible options  
D:\java>javac Lab1.java  
D:\java>java Lab1  
Prog. is showing even no  
2  
4  
6  
8  
10  
12  
14  
16  
18  
20  
D:\java>_
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 2
Abstract Classes and Inheritance

Date : _____

Ex. No. : _____

AIM: To demonstrate the use of abstract classes, inheritance, and method overriding to compute areas of different geometric shapes.

DESCRIPTION: Create an abstract Shape class with an abstract printArea() method, and implement it in subclasses Rectangle, Triangle, and Circle.

EXPERIMENT CODE:***Shape.java (Abstract Class)***

```
public abstract class Shape {
    protected int a1;
    protected int a2;
    public Shape(int d1, int d2) {
        this.a1 = d1;
        this.a2 = d2;
    }
    public abstract void printArea();
}
```

Rectangle.java

```
public class Rectangle extends Shape {
    public Rectangle(int length, int breadth) {
        super(length, breadth);
    }
    @Override
    public void printArea() {
        double area = a1 * a2;
        System.out.println("Rectangle Area is : " + area);
    }
}
```

Triangle.java

```
public class Triangle extends Shape {
    public Triangle(int base, int height) {
        super(base, height);
    }
    @Override
    public void printArea() {
        double area = 0.5 * a1 * a2;
        System.out.println("Triangle Area is : " + area);
    }
}
```


Name of the
Experiment

Date :

Ex. No. :

Circle.java

```
public class Circle extends Shape {
    public Circle(int radius) {
        super(radius, 0); // a1 is radius, a2 is unused
    }
    @Override
    public void printArea() {
        double area = Math.PI * a1 * a1;
        System.out.println("Circle Area is: "+ area);
    }
}
```

AbstractTest.java (Main Class)

```
public class AbstractTest {
    public static void main(String[] args) {
        Shape rect = new Rectangle(10, 5);
        Shape tri = new Triangle(4, 6);
        Shape circ = new Circle(7);
        System.out.println("--- Calculating Areas ---");
        rect.printArea();
        tri.printArea();
        circ.printArea();
    }
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 3
File Handling

Date :

Ex. No. :

A) Directory Traversal**AIM:** To implement recursive file system traversal to list all files and directories.**DESCRIPTION:** Use java.io.File and recursion with listFiles() to traverse and display all files in a directory tree.**EXPERIMENT CODE:**

```
import java.io.File;

public class DirectoryLister {
    public static void main(String[] args) {
        // Specify the directory path here
        File rootDir = new File("/path/to/your/directory");
        if (!rootDir.exists() || !rootDir.isDirectory()) {
            System.out.println("Invalid directory path.");
            return;
        }
        System.out.println("Listing files in: " + rootDir.getAbsolutePath());
        listAllFiles(rootDir);
    }

    public static void listAllFiles(File dir) {
        File[] files = dir.listFiles();
        if (files != null) {
            for (File file : files) {
                if (file.isDirectory()) {
                    System.out.println("Directory: " + file.getAbsolutePath());
                    // Recursive call for subdirectories
                    listAllFiles(file);
                } else {
                    System.out.println("File: " + file.getAbsolutePath());
                }
            }
        }
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

B) Random Access File**AIM:** To practice non-sequential data access using the Random Access File class.**DESCRIPTION:** Use java.io.RandomAccessFile with seek() to perform targeted read and write operations at specific file positions.**EXPERIMENT CODE:**

```

import java.io.RandomAccessFile;
import java.io.IOException;

public class RandomAccessFileDemo {
    private static final String FILE_NAME = "raf_data.txt";
    public static void main(String[] args) {
        try (RandomAccessFile raf = new RandomAccessFile(FILE_NAME, "rw")) {
            System.out.println("--- Initial Write ---");
            raf.writeUTF("Hello World"); raf.writeInt(100);
            raf.writeDouble(3.14159);
            long intPosition = raf.getFilePointer() - 8 - 4;
            System.out.println("File pointer is at: " + raf.getFilePointer());
            // --- Read Operation ---
            System.out.println("\n--- Sequential Read ---");    raf.seek(0);
            // Go back to the beginning
            System.out.println("Read String: " + raf.readUTF());
            System.out.println("Read Int: " + raf.readInt());
            System.out.println("Read Double: " + raf.readDouble());
            // --- Random Read Operation ---
            System.out.println("\n--- Random Access Read (Reading Int 100) ---");
            raf.seek(intPosition);
            // Seek to the position of the integer
            int readInt = raf.readInt();
            System.out.println("Read Int at " + intPosition + ": " + readInt);
            // --- Write Operation (Update) ---
            System.out.println("\n--- Random Access Write (Updating Int) ---");
            raf.seek(intPosition);    raf.writeInt(999);
            // Overwrite 100 with 999
            System.out.println("Int updated to 999 at " + intPosition);
            // --- Verify Update ---
            System.out.println("\n--- Verification ---");
            raf.seek(intPosition);
            System.out.println("Verified Int: " + raf.readInt());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Name of the
ExperimentLAB EXPERIMENT WEEK - 4
Multi-Threading (Random Number)

Date :

Ex. No. :

AIM: To implement a multi-threaded application where threads communicate based on a shared condition (number parity).

DESCRIPTION: Use three Thread objects: one to generate random numbers, one for square (even), and one for cube (odd) calculations.

EXPERIMENT CODE:***RandomThreadApp.java***

```
import java.util.Random;
class GeneratorThread extends Thread {
    @Override
    public void run() {
        Random random = new Random();
        try {
            for (int i = 0; i < 5; i++) {
                int randomNumber = random.nextInt(100);
                System.out.println("Random Number Generated: " + randomNumber);
                if (randomNumber % 2 == 0) {
                    new SquareThread(randomNumber).start();
                } else {
                    new CubeThread(randomNumber).start();
                }
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println("Generator thread interrupted.");
        }
    }
}

class SquareThread extends Thread {
    private int number;
    public SquareThread(int number) {
        this.number = number;
    }
    @Override
    public void run() {
        long square = (long) number * number;
        System.out.println(" Square Thread: " + square);
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
class CubeThread extends Thread {
    private int number;
    public CubeThread(int number) {
        this.number = number;
    }
    @Override
    public void run() {
        long cube = (long) number * number * number;
        System.out.println(" Cube Thread: " +cube);
    }
}

public class RandomThreadApp {
    public static void main(String[] args) {
        new GeneratorThread().start();
    }
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 5
IPC and Synchronization

Date :

Ex. No. :

A) Producer-Consumer Problem**AIM:** To implement the Producer-Consumer problem using Inter-Thread Communication mechanisms.**DESCRIPTION:** Use synchronized methods with wait() and notifyAll() on a shared buffer (queue) to coordinate data production and consumption.**EXPERIMENT CODE:**

```
import java.util.LinkedList;
import java.util.Queue;

class Buffer {
    private Queue<Integer> queue = new LinkedList<>();
    private final int CAPACITY = 5;

    // Producer method (produces data into the queue)
    public void produce(int value) throws InterruptedException {
        synchronized (this) {
            // Wait while the buffer is full
            while (queue.size() == CAPACITY) {
                System.out.println("Buffer is full. Producer waiting.");
                wait(); // Release the lock and wait
            }
            queue.add(value);
            System.out.println("Producer: " + value + ".Size: " + queue.size());
            notifyAll(); // Notify consumers that buffer is not empty
        }
    }

    // Consumer method (consumes data from the queue)
    public int consume() throws InterruptedException {
        synchronized (this) {
            // Wait while the buffer is empty
            while (queue.isEmpty()) {
                System.out.println("Buffer is empty. Consumer waiting.");
                wait(); // Release the lock and wait
            }
            int value = queue.poll();
            System.out.println("Consumer: " + value + ". Size: " +
                queue.size());
            notifyAll(); // Notify producer that buffer is not full
            return value;
        }
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
class Producer extends Thread {
    private Buffer buffer;
    public Producer(Buffer buffer) { this.buffer = buffer; }
    @Override
    public void run() {
        for (int i = 1; i <= 10; i++) {
            try {
                buffer.produce(i);
                Thread.sleep(100);
            } catch (InterruptedException e) {}
        }
    }
}

class Consumer extends Thread {
    private Buffer buffer;
    public Consumer(Buffer buffer) { this.buffer = buffer; }
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            try {
                buffer.consume();
                Thread.sleep(500);
            } catch (InterruptedException e) {}
        }
    }
}

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        Buffer buffer = new Buffer();
        Producer producer = new Producer(buffer);
        Consumer consumer = new Consumer(buffer);
        producer.start();
        consumer.start();
    }
}
```


Name of the
Experiment

Date :

Ex. No. :

B) Thread Synchronization (Reservations)

AIM: To demonstrate Thread Synchronization to ensure data integrity when multiple threads access a critical section.

DESCRIPTION: Use the synchronized keyword on the booking method to ensure only one thread can modify the shared resource (available seats) at a time.

EXPERIMENT CODE:

```
class BookingSystem {
    private int availableSeats = 5;
    // Synchronized method to ensure only one thread can book at a time
    public synchronized void bookSeat(String customerName, int seatsToBook) {
        System.out.println(customerName + " is checking availability for " +
seatsToBook + " seat(s).");
        if (availableSeats >= seatsToBook) {
            System.out.println(" " + customerName + ": Seats available. Proceeding
with booking...");
            // Simulate processing time
            try { Thread.sleep(50); } catch (InterruptedException e) {}

            availableSeats -= seatsToBook;
            System.out.println(" " + customerName + ": SUCCESSFULLY booked " +
seatsToBook + " seat(s).");
            System.out.println(" *** Remaining Seats: " + availableSeats + "
***");
        } else {
            System.out.println(" " + customerName + ": FAILED to book. Only " +
availableSeats + " seats remaining.");
        }
    }
}

class Customer extends Thread {
    private BookingSystem system;
    private String name;
    private int seatsNeeded;

    public Customer(BookingSystem system, String name, int seatsNeeded) {
        this.system = system;
        this.name = name;
        this.seatsNeeded = seatsNeeded;
    }

    @Override
    public void run() {
        System.bookSeat(name, seatsNeeded);
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
public class ReservationDemo {  
    public static void main(String[] args) {  
        BookingSystemrailBooking = new BookingSystem(); // Shared object [cite: 34]  
  
        // Multiple threads acting on the same booking system  
        Customer c1 = new Customer(railBooking, "Alice", 3);  
        Customer c2 = new Customer(railBooking, "Bob", 2);  
        Customer c3 = new Customer(railBooking, "Charlie", 2);  
  
        c1.start();  
        c2.start();  
        c3.start(); // This thread will likely fail due to lack of seats  
    }  
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 6
Applets

Date :

Ex. No. :

A) Simple Message Applet**AIM:** To develop a basic Java Applet and display a simple message on the screen.**DESCRIPTION:** Extend JApplet and use the init() method and a JLabel to display a constant message.**EXPERIMENT CODE:**

```
import javax.swing.JApplet;
import javax.swing.JLabel;
import java.awt.FlowLayout;

public class SimpleMessageApplet extends JApplet {
    // Called when the applet is initialized
    @Override
    public void init() {
        setLayout(new FlowLayout());
        JLabel message = new JLabel("Hello, this is a Simple Java Applet!");
        add(message);
    }
}
```

B) Factorial Applet**AIM:** To develop an interactive Java Applet using GUI controls and handle user input events.**DESCRIPTION:** Use JTextField for I/O and JButton with an ActionListener to compute and display the factorial of the input number.**EXPERIMENT CODE:**

```
import javax.swing.*;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class FactorialApplet extends JApplet implements ActionListener {
    private JTextField inputField;
    private JTextField outputField;
    private JButton computeButton;

    @Override
    public void init() {
        setLayout(new FlowLayout());
        add(new JLabel("Enter Integer:"));
        inputField = new JTextField(10);

        add(inputField);
        computeButton = new JButton("Compute");
        computeButton.addActionListener(this);
        add(computeButton); // Button named "Compute" is clicked
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
add(new JLabel("Factorial Value:"));
outputField = new JTextField(15);
outputField.setEditable(false);
add(outputField);
}

@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == computeButton) {
        try {
            int n = Integer.parseInt(inputField.getText());
            if (n < 0 || n > 20) {
                outputField.setText("Input must be 0-20");
            } else {
                int fact = calculateFactorial(n);
                outputField.setText(String.valueOf(fact));
            }
        } catch (NumberFormatException ex) {
            outputField.setText("Invalid Integer Input");
        }
    }
}

// Recursive factorial function
private long calculateFactorial(int n) {
    if (n == 0) {
        return 1;
    }
    return n * calculateFactorial(n - 1);
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 7
GUI - Simple Calculator

Date :

Ex. No. :

AIM: To create a GUI-based simple calculator and implement basic arithmetic operations with exception handling.

DESCRIPTION: Use JFrame with GridLayout for buttons, JTextField for display, and try-catch blocks to handle division by zero.

EXPERIMENT CODE:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class CalcDemo extends JFrame implements ActionListener{
    JTextField tf = new JTextField();
    int num1 = 0, num2 = 0;
    double result;
    String op = "";

    CalcDemo(){
        super("Calculator Demonstration...");
        GridBagLayout gb = new GridBagLayout();
        setLayout(gb);
        GridBagConstraints gc = new GridBagConstraints();
        tf.setFont(new Font("Arial", Font.BOLD, 20));
        gc.gridx = 0;
        gc.gridy = 0;
        gc.gridwidth = 4;
        gc.insets = new Insets(5,5,5,5);
        gc.fill = GridBagConstraints.HORIZONTAL;
        add(tf,gc);
        gc.gridwidth = 1; // Reset
        String[] ss = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "+",
            "0", "-", "C", "=" };
        int x = 0, y = 1;
        for(String s : ss){
            JButton b = new JButton(s);
            b.addActionListener(this);

            gc.ipadx = 30;
            gc.ipady = 30;
            gc.gridx = x;
            gc.gridy = y;

            gc.insets = new Insets(5,5,5,5);
            gc.fill = GridBagConstraints.BOTH;
            add(b, gc);
            x++;
        }
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```

        if (x == 4) {
            x = 0; y++;
        }
    }
    setSize(400,500);
    setVisible(true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
public void actionPerformed(ActionEvent ae){
    String s = ae.getActionCommand();
    if(s.charAt(0)>= '0' && s.charAt(0) <='9'){
        tf.setText(tf.getText()+s);
    }else if(s.equals("C")){
        num1 = num2 = 0; op = ""; result = 0.0;
        tf.setText("");
    }else if(s.equals("=")){
        num2 = Integer.parseInt(tf.getText());
        switch(op){
            case "+" : result = num1 + num2; break;
            case "-" : result = num1 - num2; break;
            case "*" : result = num1 * num2; break;
            case "/" :
                if(num2 == 0.0){
                    tf.setText("Error");return;
                }else{
                    result =(double)num1/(double)num2;
                    break;
                }
            }
        tf.setText(String.valueOf(result));
    }else{
        num1 = Integer.parseInt(tf.getText());
        op = s;
        tf.setText("");
    }
}
public static void main(String...args){
    new CalcDemo();
}
}

```

Name of the
ExperimentLAB EXPERIMENT WEEK - 8
GUI - Traffic Light Simulation

Date :

Ex. No. :

AIM: To simulate a traffic light using radio buttons and update a label based on user selection.

DESCRIPTION: Use JRadioButtons in a ButtonGroup and an ActionListener to change the text and color of a JLabel based on the selection.

EXPERIMENT CODE:

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class TrafficSignal extends JFrame implements ActionListener {
    JRadioButton rb;
    JLabel l;
    Panel p;
    int a = 0;

    TrafficSignal(){
        super("Traffic Lights Demo");
        BorderLayout bl = new BorderLayout();
        setLayout(bl);
        p = new Panel();
        add(p, bl.NORTH);

        GridBagLayout gb = new GridBagLayout();
        p.setLayout(gb);
        GridBagConstraints gc = new GridBagConstraints();

        gc.fill = GridBagConstraints.HORIZONTAL;
        String[] c = {"Red", "Yellow", "Green"};
        int x = 0, y = 0;
        ButtonGroup bg = new ButtonGroup();
        for(String s : c){
            rb = new JRadioButton(s);
            rb.addActionListener(this);
            gc.gridx = x;
            gc.gridy = y;
            p.add(rb, gc);
            bg.add(rb);
            x++;
        }
        l = new JLabel("GET SET TO GO");
        p = new Panel();
        add(p, bl.SOUTH);
        p.add(l);
        setSize(400, 500);

        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

Name of the
Experiment

Date :

Ex. No. :

```

public void actionPerformed(ActionEvent ae){

    String cc = ae.getActionCommand();
    if(cc.equals("Red")){
        a = 1; repaint();
        l.setText("! STOP !");
    }else if(cc.equals("Yellow")){
        a = 2; repaint();
        l.setText("SLOW DOWN");
    }else{
        a = 3; repaint();
        l.setText("GO ----> ");
    }
}

public void paint(Graphics g){
    g.setColor(Color.BLACK);
    g.fillRect(100, 105, 110, 270);
    g.setColor(Color.WHITE);
    g.fillOval(125, 150, 60, 60);
    g.fillOval(125, 230, 60, 60);
    g.fillOval(125, 300, 60, 60);

    switch(a){
        case 1:
            g.setColor(Color.RED);
            g.fillOval(125, 150, 60, 60);
            break;
        case 2:
            g.setColor(Color.YELLOW);
            g.fillOval(125, 230, 60, 60);
            break;
        case 3:
            g.setColor(Color.GREEN);
            g.fillOval(125, 300, 60, 60);
        default: l.setText("GET SET GO");
    }
}

public static void main(String args[]){
    new TrafficSignal();
}
}

```


Name of the
ExperimentLAB EXPERIMENT WEEK - 9
GUI - Integer Division with Exception Handling

Date :

Ex. No. :

AIM: To implement robust exception handling in a GUI for input validation and arithmetic operations.

DESCRIPTION: Use a try-catch block to specifically handle NumberFormatException and ArithmeticException, displaying errors via JOptionPane.

EXPERIMENT CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class ExceptionDivision extends JFrame implements ActionListener {
    private JTextField tf1, tf2, tf3;
    private JLabel l;
    private JButton btn;

    public ExceptionDivision() {
        super("Integer Division");
        GridLayout gl = new GridLayout(4, 2);
        setLayout(gl);

        l = new JLabel("Enter Value 1");
        add(l);
        tf1 = new JTextField();
        tf1.addActionListener(this);
        add(tf1);

        l = new JLabel("Enter Value 2");
        add(l);
        tf2 = new JTextField();
        tf2.addActionListener(this);
        add(tf2);

        l = new JLabel("Result");
        add(l);
        tf3 = new JTextField();
        tf3.addActionListener(this);
        add(tf3);

        l = new JLabel();
        add(l);
        btn = new JButton("DIVIDE");
        btn.addActionListener(this);
        add(btn);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
        setSize(400, 400);
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
@Override
public void actionPerformed(ActionEvent ae) {
    String cc = ae.getActionCommand();
    if(!cc.equals("")){
        tf3.setText("");
        try{
            int v1 = Integer.parseInt(tf1.getText());
            int v2 = Integer.parseInt(tf2.getText());

            int result = v1/v2;
            String output = "Division is: "+result;
            tf3.setText("Division is:"+String.valueOf(result));
            JOptionPane.showMessageDialog(this, output, "OUTPUT",
JOptionPane.PLAIN_MESSAGE);
        }catch(Exception e){
            JOptionPane.showMessageDialog(this, e.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        new ExceptionDivision();
    }
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 10
Mouse Events (Adapter Classes)

Date :

Ex. No. :

AIM: To demonstrate handling various Mouse Events using Adapter Classes.

DESCRIPTION: Extend MouseAdapter (and MouseMotionAdapter) to capture and display the names of all mouse events fired at the center of the window.

EXPERIMENT CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class MouseAdapterDemo extends JFrame {
    private JLabel l;
    public MouseAdapterDemo() {
        setTitle("Mouse Events Demo (Adapter)");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        l = new JLabel("Move or Click the Mouse", SwingConstants.CENTER);
        l.setFont(new Font("Arial", Font.BOLD, 16));
        add(l, BorderLayout.CENTER);

        this.addMouseListener(new CustomMouseListener());
        this.addMouseMotionListener(new CustomMouseMotionAdapter());
        setVisible(true);
    }
    // Custom MouseAdapter class
    private class CustomMouseListener extends MouseAdapter {
        @Override
        public void mouseClicked(MouseEvent e) {
            updateLabel("Mouse Clicked");
        }
        @Override
        public void mousePressed(MouseEvent e) {
            updateLabel("Mouse Pressed");
        }
        @Override
        public void mouseReleased(MouseEvent e) {
            updateLabel("Mouse Released");
        }
        @Override
        public void mouseEntered(MouseEvent e) {
            updateLabel("Mouse Entered Window");
        }
        @Override
        public void mouseExited(MouseEvent e) {
            updateLabel("Mouse Exited Window");
        }
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
// Custom MouseMotionAdapter class

private class CustomMouseMotionAdapter extends MouseAdapter {
    @Override
    public void mouseMoved(MouseEvent e) {
        updateLabel("Mouse Moved at (" + e.getX() + ", " + e.getY() + ")");
    }
    @Override
    public void mouseDragged(MouseEvent e) {
        updateLabel("Mouse Dragged at (" + e.getX() + ", " + e.getY() +
    "));
    }
}

private void updateLabel(String eventName){
    l.setText(eventName);
}

public static void main(String[] args) {
    new MouseAdapterDemo();
}
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 11
GUI - Displaying a Table

Date :

Ex. No. :

AIM: To read and parse comma-separated data from a text file and display it in a structured table format using GUI components.

DESCRIPTION: Read the .txt file line by line, split by comma, and display the data using Labels organized by Grid Layout.

EXPERIMENT CODE:

Table.txt (Create this file in the project directory)

Name, Age, City
Shafakhat, 30, New York
Mohammed, 24, London
Khan, 45, Paris

TableDisplay.java

```
import javax.swing.*;
import java.awt.*;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class TableDisplay extends JFrame {
    private static final String FILE_NAME = "Table.txt";

    public TableDisplay() {
        setTitle("Table Data Display");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        List<String[]> data = readTableData();
        if (data.isEmpty()) {
            add(new JLabel("Could not read data from " + FILE_NAME));
        } else {
            int rows = data.size();
            int cols = data.get(0).length;

            setLayout(new GridLayout(rows, cols, 5, 5));

            for (int i = 0; i < rows; i++) {
                String[] rowData = data.get(i);
                for (int j = 0; j < cols; j++) {
                    JLabel label = new JLabel(rowData[j], SwingConstants.CENTER);

                    label.setBorder(BorderFactory.createLineBorder(Color.GRAY));

                    if (i == 0) {
                        label.setFont(label.getFont().deriveFont(Font.BOLD));
                    }
                }
            }
        }
    }

    private List<String[]> readTableData() {
        List<String[]> data = new ArrayList<>();
        try {
            BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME));
            String line;
            while ((line = reader.readLine()) != null) {
                String[] row = line.split(",");
                data.add(row);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return data;
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```

        label.setBackground(Color.LIGHT_GRAY);
        label.setOpaque(true);
    }
    add(label);
}
}
}
setVisible(true);
setSize(400, 400);
}
private List<String[]>readTableData() {
    List<String[]> data = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] values = line.split(",");
            data.add(values);
        }
    } catch (IOException e) {

        JOptionPane.showMessageDialog(this,e.getMessage(),"ERROR",JOptionPane.
        e.ERROR_MESSAGE);
    }
    return data;
}
public static void main(String[] args) {
    new TableDisplay();
}
}

```

Name of the
ExperimentLAB EXPERIMENT WEEK - 12
Collections - Reading Data into a Hash Table

Date :

Ex. No. :

AIM: To use the Hashtable collection class to store and retrieve data based on bidirectional key-value lookup.

DESCRIPTION: Read tab-separated Name/Phone pairs into two Hashtables (Name ↔ Phone) to perform lookups based on user input.

EXPERIMENT CODE:

Data.txt (Create this file in the project directory)

```
Shafakhat    7032-6543
Khan    9885-4677
Mohammed    8686-5599
```

HashTableLookup.java

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.Hashtable;
import java.util.Scanner;

public class HashTableLookup {
    private static final String FILE_NAME = "Data.txt";
    private static Hashtable<String, String>nameToPhone = new Hashtable<>();
    private static Hashtable<String, String>phoneToName = new Hashtable<>();

    public static void main(String[] args) {
        if (!loadData()) {
            System.out.println("Exiting program.");
            return;
        }
        System.out.println("\n--- Data Loaded. Ready for Lookup ---");
        try (Scanner scanner = new Scanner(System.in)) {
            while (true) {
                System.out.print("Enter Name or Phone Number (or 'exit'): ");
                String input = scanner.nextLine().trim();

                if (input.equalsIgnoreCase("exit"))
                    break;

                if (input.matches(".*[0-9].*") || input.startsWith("555")) {
                    String name = phoneToName.get(input);
                    if (name != null)
                        System.out.println("Corresponding Name: " + name);
                    else
                        System.out.println("Phone number not found.");
                } else {
                    String phone = nameToPhone.get(input);
```

Name of the
Experiment

Date :

Ex. No. :

```

        if (phone != null) {
            System.out.println("Corresponding Phone: " + phone);
        } else {
            System.out.println("Name not found.");
        }
    }
}

private static boolean loadData() {
    try (BufferedReader br = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.split("\t");
            if (parts.length == 2) {
                String name = parts[0].trim();
                String phone = parts[1].trim();
                nameToPhone.put(name, phone);
                phoneToName.put(phone, name);
            }
        }
        return true;
    } catch (IOException e) {
        System.err.println("Error reading file: " + e.getMessage());
        return false;
    }
}
}

```


Name of the
ExperimentLAB EXPERIMENT WEEK - 13
Data Structures - Doubly Linked List

Date :

Ex. No. :

AIM: To implement the core operations (Create, Delete, Display) of a Doubly Linked List data structure.

DESCRIPTION: Manually implement a Doubly Linked List with Node objects containing prev and next pointers to manage element creation and deletion.

EXPERIMENT CODE:

```
import java.util.LinkedList;

public class DoublyLinkedListDemo {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        System.out.println("--- i. Create (Insertion using add/addLast) ---");
        list.add(10);
        list.addLast(20);
        list.addFirst(5);
        list.add(2, 15);
        System.out.println(list);

        System.out.println("\n--- ii. Delete a given element (Removal) ---");

        int elementToDelete = 20;
        boolean wasRemoved = list.remove(Integer.valueOf(elementToDelete));

        if(wasRemoved) {
            System.out.println("Deleted element: " + elementToDelete);
        } else {
            System.out.println("Element " + elementToDelete + " not found.");
        }

        System.out.println(list);

        list.removeFirst();
        System.out.println("Deleted head element.");

        System.out.println("\n--- iii. Display after Deletion ---");
        System.out.println(list);

        System.out.println("\n--- Deleting non-existent element ---");
        list.remove(Integer.valueOf(100));
        System.out.println("Element 100 was not found for deletion.");
        System.out.println(list);
    }
}
```

Name of the
ExperimentLAB EXPERIMENT WEEK - 14
Database Connectivity (JDBC)

Date :

Ex. No. :

AIM: To demonstrate Database Connectivity (JDBC) by performing CRUD operations on a database table.

DESCRIPTION: Use the java.sql package with Connection, PreparedStatement, and ResultSet to Create, Read, Update, and Delete records in a student table.

EXPERIMENT CODE:

```
import java.sql.*;

public class JDBCCrudDemo {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            try (Connection conn = DriverManager.getConnection(JDBC_URL, USER,
PASSWORD)) {
                System.out.println("Database connection established.");
                createTable(conn);
                // CRUD Operations
                // C - Create (Insert)
                createStudent(conn, 1, "Alice", 20);
                createStudent(conn, 2, "Bob", 22);
                // R - Read (Select All)
                readAllStudents(conn);
                // U - Update
                updateStudentName(conn, 1, "Alice Smith");
                // R - Read (Select After Update)
                readAllStudents(conn);
                // D - Delete
                deleteStudent(conn, 2);
                // R - Read (Select After Delete)
                readAllStudents(conn);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        } catch (ClassNotFoundException e) {
            System.err.println("JDBC Driver not found.");
            e.printStackTrace();
        }
    }

    private static void createTable(Connection conn) throws SQLException {
        String sql = "CREATE TABLE IF NOT EXISTS student ("
            + "id INT PRIMARY KEY," + "name VARCHAR(100) NOT NULL," + "age INT)";
        try (Statement stmt = conn.createStatement()) {
            stmt.executeUpdate(sql);
            System.out.println("\nTable 'student' checked/created.");
        }
    }
}
```

Name of the
Experiment

Date :

Ex. No. :

```
// C - Create (Insert) operation
private static void createStudent(Connection conn, int id, String name, int
age) throws SQLException {
    String sql = "INSERT INTO student (id, name, age) VALUES (?, ?, ?)";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        pstmt.setString(2, name);
        pstmt.setInt(3, age);
        int rowsAffected = pstmt.executeUpdate();
        System.out.println("\nCreated or row(s) inserted for " + name);
    }
}

// R - Read (Select All) operation
private static void readAllStudents(Connection conn) throws SQLException {
    String sql = "SELECT id, name, age FROM student";
    System.out.println("\n--- All Students ---");
    try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {
        while (rs.next()) {
            int id = rs.getInt("id");
            String name = rs.getString("name");
            int age = rs.getInt("age");
            System.out.printf("ID: %d, Name: %s, Age: %d\n",
                id, name, age);
        }
    }
}

// U - Update operation
private static void updateStudentName(Connection conn, int id, String newName)
throws SQLException {
    String sql = "UPDATE student SET name = ? WHERE id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, newName);
        pstmt.setInt(2, id);
        int rowsAffected = pstmt.executeUpdate();
        System.out.println("\nUpdated row(s) updated. ID: " + id);
    }
}

// D - Delete operation
private static void deleteStudent(Connection conn, int id) throws
SQLException{
    String sql = "DELETE FROM student WHERE id = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        int rowsAffected = pstmt.executeUpdate();
        System.out.println("\nDeleted rows for ID"+id);
    }
}
}
```